

SISTEMA BANCÁRIO: UMA ABORDAGEM EM PROGRAMAÇÃO ORIENTADA A OBJETOS

Thiago C. de Castro¹, Abraham D. W. B. Adande¹, Silas L. S. de Souza¹

¹Universidade Federal do Pará

Faculdade de Computação – Instituto de Ciências Exatas e Naturais

Av. Augusto Correa 01, 66075-090 – Belém – PA – Brasil

thiago.correa@icen.ufpa.br, abraham.adande@icen.ufpa.br, silascou@icen.ufpa.br

Resumo. Este relatório descreve o desenvolvimento e a arquitetura de um Sistema Bancário construído em Java, concebido para consolidar conceitos avançados de Programação Orientada a Objetos (POO). O projeto estrutura-se em torno de modelos de domínio essenciais, incluindo a gestão de clientes e a implementação de múltiplos tipos de contas bancárias, como Conta Corrente, Conta Poupança e Conta de Investimento. Um diferencial técnico da aplicação é a adoção de Padrões de Projeto (Design Patterns) para resolver problemas de arquitetura: o padrão Factory é utilizado para a instanciação das contas, enquanto o padrão Strategy gerencia a lógica de cobrança de tarifas, permitindo operações com taxas fixas, percentuais ou isentas. Adicionalmente, o sistema garante a integridade das operações através de tratamento de exceções customizadas (como validação de contas e saldo insuficiente) e emprega persistência de dados em arquivos CSV, complementado por um módulo de logging e uma interface de usuário interativa via terminal.

Abstract. This report details the development and architecture of a Java-based Banking System, designed to consolidate advanced Object-Oriented Programming (OOP) concepts. The project is structured around essential domain models, including client management and the implementation of multiple bank account types, such as Checking, Savings, and Investment accounts. A key technical highlight of the application is the adoption of Design Patterns to solve architectural challenges: the Factory pattern is used for account instantiation, while the Strategy pattern manages the fee collection logic, enabling transactions with fixed, percentage-based, or exempt rates. Furthermore, the system ensures transactional integrity through custom exception handling (such as account validation and insufficient funds) and employs data persistence using CSV files, complemented by a logging module and an interactive command-line user interface.

Sumário

1	Introdução e Objetivo	3
1.1	Introdução	3
1.2	Objetivo Geral	3
1.3	Escopo do Projeto	3
2	Modelagem de Classes e Arquitetura	4
2.1	Descrição dos Pacotes	4
2.2	Entidades de Domínio	4
2.3	Herança e Polimorfismo	4
2.4	Interfaces	5
3	Padrões de Projeto	6
3.1	Factory Method	6
3.2	Strategy	6
3.3	Singleton	6
4	Lógica de Negócio e Tratamento de Exceções	8
4.1	Separação de Responsabilidades (SRP)	8
4.2	Segurança e Robustez	8
5	Persistência de Dados	9
5.1	Mecanismo de Armazenamento	9
5.2	Ficheiros Utilizados	9
6	Evidências de Funcionalidade e Interação	10
6.1	Interface do Utilizador (GUI)	10
6.2	Capturas de Ecrã (Prints) e Validação de Regras de Negócio	10
6.2.1	Acesso e Gestão Inicial de Contas	10
6.2.2	Transações Bem-Sucedidas (Depósitos e Transferências)	13
6.2.3	Bloqueio de Operações Inválidas e Segurança	15
7	Diagrama UML	17
8	Conclusão	18

1. Introdução e Objetivo

1.1. Introdução

O presente relatório documenta o desenvolvimento de uma aplicação prática para a disciplina de Paradigmas de Linguagem de Programação (PLP), do curso de Bacharelado em Ciência da Computação da Universidade Federal do Pará (UFPA). Em conformidade com a proposta da "Opção 2 - Aplicação Livre Orientada a Objetos" exigida na avaliação, o domínio escolhido para o projeto foi a simulação de um Sistema Bancário.

1.2. Objetivo Geral

O objetivo principal deste projeto consiste em criar um Sistema Bancário completo, funcional e robusto, com o propósito de aplicar e consolidar os conceitos da Programação Orientada a Objetos (POO).

A aplicação foi projetada para ir além dos requisitos mínimos da disciplina — como a criação de classes, encapsulamento, instanciamento e estruturas de controle —, buscando explorar e demonstrar a aplicação de conceitos avançados. Isso inclui a utilização de Herança, Polimorfismo, Classes Abstratas e Interfaces, bem como a implementação de Padrões de Projeto consagrados (como Factory Method, Strategy e Singleton) para resolver problemas complexos de design e garantir a flexibilidade das regras de negócio da instituição financeira.

1.3. Escopo do Projeto

O sistema foi integralmente desenvolvido na linguagem de programação Java (utilizando a versão 26.0.1). Para assegurar a qualidade do software, a facilidade de manutenção e o baixo acoplamento, o código-fonte foi estruturado e gerido numa arquitetura de pacotes altamente coesa (br.com.sistemabancario).

O escopo técnico do projeto abrange as seguintes características fundamentais:

- **Separação de Responsabilidades (SRP):** Divisão rigorosa da aplicação em pacotes especializados, isolando Entidades de Domínio (modelo), Contratos (interfaces), Padrões de Projeto (padroes), Regras e Gestão de Negócio (gerenciamento), Tratamento de Erros (excecoes) e Interface de Usuário (ui).
- **Persistência de Dados (Bônus):** Implementação de armazenamento não volátil (pacote persistencia), garantindo que o estado das contas, o histórico de transações (arquivos .csv) e os registros de eventos do sistema (arquivo .log) sejam salvos e recuperados com segurança fora da memória da aplicação.
- **Validação e Segurança:** Tratamento rigoroso de exceções (incluindo falhas customizadas como SaldoInsuficiente e ContaInvalida) e validação de entradas do usuário através da interface gráfica/interativa, impedindo travamentos e garantindo a estabilidade da execução.

2. Modelagem de Classes e Arquitetura

A arquitetura do Sistema Bancário foi projetada com foco em alta coesão e baixo acoplamento, seguindo rigorosamente o Princípio da Responsabilidade Única (SRP) do SOLID. O código-fonte está estruturado em pacotes lógicos, o que facilita a manutenção, a escalabilidade e o entendimento do domínio.

2.1. Descrição dos Pacotes

O projeto, contido no pacote raiz `br.com.sistemabancario`, divide-se nas seguintes frentes de responsabilidade:

- **modelo:** Contém as Entidades de Domínio (POJOs). É o núcleo de dados do sistema, onde residem a superclasse `Conta`, suas derivadas e a classe `Cliente`. Representa o estado do negócio.
- **interfaces:** Subpacote dedicado à definição de contratos abstratos. Garante o desacoplamento ao estipular regras de comportamento que podem ser adotadas por diferentes classes.
- **padroes:** Isola as implementações específicas de Padrões de Projeto (Design Patterns), como as lógicas de criação (Factory) e de definição de taxas (Strategy).
- **excecoes:** Encapsula o tratamento de erros customizados vinculados a regras de negócio, como `SaldoInsuficiente` e `ContaInvalida`, promovendo maior robustez.
- **gerenciamento:** Responsável pela lógica de controle central. Contém classes como `Banco` e `GerenciadorTransacoes`, retirando das contas a responsabilidade de gerenciar operações complexas que envolvam múltiplas entidades.
- **persistencia:** Módulo de I/O encarregado de interagir com o sistema de arquivos, gravando e recuperando informações (contas e logs) em arquivos físicos (CSV e TXT).
- **ui:** Representa a camada de visualização (User Interface). Abriga a classe `Main` para inicialização limpa do sistema e o `MenuInterativo`, que gerencia o loop de interação e a captura de inputs do usuário.

2.2. Entidades de Domínio

As entidades principais formam a base estrutural de dados da aplicação:

- **Cliente:** Responsável por armazenar os dados sensíveis e de identificação do usuário, como nome, cpf e senha. Possui uma relação de agregação/composição com as contas, indicando a titularidade.
- **Conta:** Superclasse base do sistema financeiro. Centraliza atributos comuns como saldo, numero da conta e a referência ao titular (Cliente). Além disso, encapsula uma lista de objetos `Transacao`, compondo o histórico necessário para a geração de extratos.

2.3. Herança e Polimorfismo

O sistema utiliza os pilares da herança e polimorfismo para tratar as particularidades dos diferentes produtos bancários oferecidos, promovendo o reaproveitamento de código (DRY - Don't Repeat Yourself):

- Herança: A classe Conta foi modelada como abstrata («abstract»), pois não faz sentido no domínio do negócio instanciar uma "Conta" genérica. A partir dela, derivam-se as classes filhas ContaCorrente (que implementa a lógica de um limite de cheque especial), ContaPoupanca (que incorpora o conceito de rendimento) e ContaInvestimento (que introduz um fator de risco). As filhas herdam toda a estrutura base de saldo e número, estendendo-a com suas especificidades.
- Polimorfismo: A manifestação principal do polimorfismo ocorre na sobrescrita (anotação @Override) do método sacar(double valor). Enquanto a assinatura do método é definida na superclasse, o comportamento em tempo de execução varia: um saque na ContaCorrente pode abater do limite de crédito e cobrar uma taxa de operação, ao passo que a ContaPoupanca realiza apenas a validação restrita ao saldo disponível atual.

2.4. Interfaces

O uso de interfaces foi aplicado para definir contratos de comportamento flexíveis, favorecendo o Princípio do Aberto/Fechado (OCP):

- Autenticavel: Interface concebida para gerenciar o processo de login através do método autenticar(String senha). O uso de uma interface em vez de um método direto no Cliente permite a expansão futura do sistema. Por exemplo, se futuramente for criada uma classe Gerente ou Funcionario, eles poderão assinar esse mesmo contrato de autenticação sem precisarem herdar de Cliente.
- ITaxaOperacao: Pilar do padrão de projeto Strategy. Ela define o contrato calcularTaxa(double valor). A adoção dessa interface justifica-se por remover a responsabilidade de calcular tarifas de dentro das classes de conta. Assim, as lógicas de tarifação (TaxaFixa, TaxaPercentual ou IsentoTaxa) são implementadas separadamente, permitindo que a regra de cobrança de uma conta seja trocada dinamicamente sem a necessidade de alterar a sua classe estrutural.

3. Padrões de Projeto

Para resolver problemas recorrentes de arquitetura de software e garantir que o Sistema Bancário permaneça escalável, flexível e de fácil manutenção, o desenvolvimento incorporou três Padrões de Projeto (Design Patterns) clássicos sugeridos pelo GoF (Gang of Four). Cada padrão foi estrategicamente posicionado em pacotes específicos (padroes.fabrica e padroes.estrategia) ou aplicado diretamente na gestão da aplicação.

3.1. Factory Method

O padrão Factory Method foi adotado para encapsular e centralizar a complexa lógica de criação de objetos da hierarquia de contas. No sistema, a classe ContaFabrica atua como a única responsável por instanciar os diferentes tipos concretos de conta, como ContaCorrente, ContaPoupanca e ContaInvestimento.

Essa abordagem fornece um alto grau de desacoplamento. A interface de usuário (UI) e as classes de gerenciamento não precisam conhecer os detalhes construtivos (ou a palavra-chave new) de cada subclasse específica de conta. Ao passar apenas o tipo de conta desejado e os dados do Cliente titular para a ContaFabrica, a fábrica devolve a abstração Conta já devidamente inicializada. Isso garante que a adição futura de novos tipos de contas (como uma "Conta Universitária") exija modificações apenas na fábrica, isolando o restante do código.

3.2. Strategy

O padrão Strategy foi implementado para gerenciar e flexibilizar as políticas de cobrança de tarifas bancárias, retirando o "peso" matemático dessa responsabilidade das próprias classes de conta. Esse padrão se estrutura a partir da interface ITaxaOperacao, que estabelece um contrato universal chamado calcularTaxa(double valor).

A partir dessa interface, foram criadas implementações concretas que representam diferentes famílias de algoritmos tarifários:

- TaxaFixa: Aplica um valor constante sobre a operação (ex: R\$ 5,00 por transferência).
- TaxaPercentual: Calcula a tarifa com base em uma porcentagem do montante transacionado.
- IsentoTaxa: Retorna um valor nulo, representando operações gratuitas para contas ou clientes premium.

A principal vantagem dessa arquitetura é permitir que a estratégia de cobrança seja substituída ou alterada dinamicamente em tempo de execução, sem necessidade de modificar a estrutura da classe de conta em si, respeitando de forma estrita o Princípio Aberto/Fechado (OCP) do SOLID.

3.3. Singleton

Para assegurar a integridade do estado da aplicação e garantir uma coordenação centralizada, o padrão Singleton foi aplicado à classe Banco. O Banco funciona como o repositório principal em memória do sistema, abrigando as listas globais de instâncias de Cliente e Conta.

Se o sistema permitisse a instanciação de múltiplos objetos Banco, as informações ficariam fragmentadas, correndo o risco de uma transação ser efetuada em um escopo que não reconhece as contas cadastradas na interface do usuário. Para evitar isso, a classe Banco possui seu construtor definido como private e expõe a própria instância única através de um método estático (getInstancia()). Assim, tanto os módulos de gerenciamento de transações, quanto o menu interativo e o sistema de persistência, acessam e modificam exatamente a mesma base de dados global em memória durante toda a execução da aplicação.

4. Lógica de Negócio e Tratamento de Exceções

Para garantir a estabilidade do Sistema Bancário e a correta aplicação das regras financeiras, a lógica de negócio foi arquitetada com foco na manutenibilidade e na prevenção de falhas, aplicando princípios de design limpo e tratamento rigoroso de erros.

4.1. Separação de Responsabilidades (SRP)

Em modelagens iniciais de sistemas orientados a objetos, é comum que operações complexas, como uma transferência, sejam alocadas diretamente dentro da entidade base (neste caso, a classe Conta). Contudo, em estrita observância ao Princípio da Responsabilidade Única (SRP - Single Responsibility Principle), essa arquitetura foi refatorada com a introdução da classe GerenciadorTransacoes.

O papel do GerenciadorTransacoes é atuar como um mediador especializado, isolando a lógica financeira de transferências. Uma transferência exige a coordenação simultânea entre duas entidades distintas (a conta de origem e a conta de destino), validação de fundos, aplicação dinâmica de tarifas (através do padrão Strategy) e o registro da operação nos extratos de ambos os envolvidos. Ao delegar essa orquestração ao gerenciador, evita-se que a classe Conta assuma responsabilidades excessivas (evitando o anti-padrão God Class), mantendo-a altamente coesa e focada exclusivamente em gerir seu próprio estado e propriedades básicas.

4.2. Segurança e Robustez

Tratando-se de um domínio financeiro, o sistema deve ser resiliente a entradas imprevistas e violações de regras de negócio. Para prevenir interrupções abruptas da aplicação (crashes), o desenvolvimento implementou um controle de fluxo de exceções robusto em múltiplas camadas:

- **Exceções Customizadas:** Para dar semântica aos erros de negócio, foram desenvolvidas classes de exceções próprias, como SaldoInsuficiente e ContaInvalida. A exceção SaldoInsuficiente é lançada pela camada de modelo sempre que uma operação de débito (saque ou transferência) excede a soma do saldo atual com o limite de crédito (se houver). Por sua vez, a ContaInvalida é acionada ao tentar realizar operações com números de conta inexistentes no repositório central. Isso permite que a aplicação identifique exatamente qual regra do banco foi violada.
- **Blocos Try-Catch e Tratamento de Input:** Na camada de interface do usuário (MenuInterativo), a interação é envelopada por blocos try-catch. Essas estruturas são responsáveis por capturar tanto as exceções customizadas de negócio quanto as exceções nativas do Java, como a InputMismatchException (gerada quando o usuário insere caracteres alfabéticos em campos de valores numéricos de depósito). Ao interceptar o erro, o sistema limpa o buffer de leitura, previne a quebra da aplicação e exibe um alerta amigável e instrutivo no terminal. Dessa forma, o loop principal de execução permanece ativo e o sistema se mantém íntegro do início ao fim do uso.

5. Persistência de Dados

5.1. Mecanismo de Armazenamento

O Sistema Bancário implementa um mecanismo de armazenamento não volátil concebido para assegurar que o estado das contas, o histórico de transações e os registos de eventos sejam salvos e recuperados de forma segura, persistindo fora da memória volátil da aplicação. Para garantir a separação de responsabilidades (SRP) na arquitetura, toda a manipulação de leitura e gravação foi encapsulada no módulo especializado persistencia. Neste pacote, a classe GerenciadorArquivo é a principal entidade encarregue de interagir com o sistema de ficheiros. As operações de Input/Output (I/O) são orquestradas através do método salvarDados(List<Conta> contas) para persistir a informação atual, e pelo método carregarDados() para instanciar e reconstruir a lista de contas na memória durante o arranque do banco.

5.2. Ficheiros Utilizados

Os dados gerados e manipulados pelo sistema ficam isolados do código-fonte, armazenados num diretório físico dedicado designado por dados/. O sistema recorre a formatos CSV (valores separados por vírgulas) e texto estruturado para organizar a informação em três ficheiros fundamentais:

- `contas_salvas.csv`: É o ficheiro central destinado à gravação e leitura de todas as contas bancárias registadas na aplicação. Este ficheiro garante que a integridade dos saldos, limites e titulares das contas seja preservada entre sessões distintas.
- `historico_transacoes.csv`: Ficheiro responsável pela persistência do histórico financeiro contínuo do sistema bancário. Funciona como uma base de dados dos extratos, armazenando e rastreando cronologicamente todas as transações realizadas (como levantamentos, depósitos ou transferências).
- `sistema.log`: Ficheiro de texto contínuo gerado e gerido automaticamente pela classe `LoggerSistema`. A sua função exclusiva é monitorizar e registar a atividade técnica da aplicação, guardando tanto avisos operacionais como erros processados em tempo real de execução, utilizando métodos de captura de exceções como o `registrarErro`.

6. Evidências de Funcionalidade e Interação

6.1. Interface do Utilizador (GUI)

Para garantir uma experiência de uso imersiva e acessível, o sistema bancário foi dotado de uma Interface Gráfica do Utilizador (GUI) moderna, substituindo a tradicional interação por consola (CLI). Esta interface atua como o mediador principal (MenuInterativo), permitindo uma navegação contínua e dinâmica.

Através deste ambiente, o utilizador pode autenticar-se com segurança, abrir novas contas (despoletando a instanciação em retaguarda através da classe ContaFabrica) e aceder a um painel central de gestão financeira. O loop de interação do sistema mantém-se ativo ininterruptamente, permitindo ao utilizador alternar entre diferentes contas e executar múltiplas operações (como depósitos, levantamentos e transferências) até que este decida encerrar a sessão deliberadamente através da opção "Sign out". Toda a complexidade da Programação Orientada a Objetos — incluindo as heranças, polimorfismo e cálculos de taxas — é abstraída, proporcionando uma experiência de uso fluida.

6.2. Capturas de Ecrã (Prints) e Validação de Regras de Negócio

Abaixo descrevem-se e apresentam-se as evidências visuais do funcionamento correto do sistema, que comprovam a integração bem-sucedida entre a interface de utilizador e a lógica de negócio estruturada no Back-end.

6.2.1. Acesso e Gestão Inicial de Contas

A aplicação apresenta inicialmente uma página de apresentação concisa do sistema bancário OOP (Figura 1). De seguida, o utilizador é encaminhado para os ecrãs de autenticação ou criação de um novo perfil (Figuras 2 e 3). Uma vez validado, o sistema carrega o painel de contas (Accounts). Como ilustrado nas Figuras 4 e 5, as contas previamente instanciadas ou recuperadas pela camada de persistência são listadas individualmente, exibindo o titular, o número parcial da conta e o saldo atual.

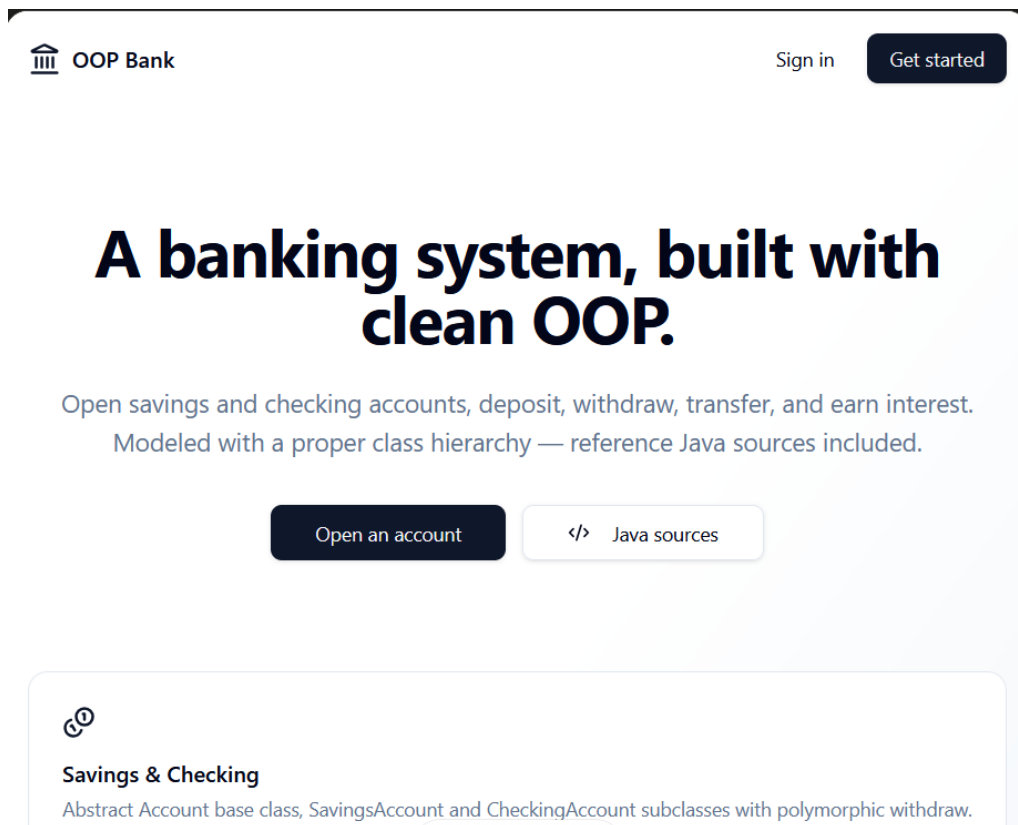


Figura 1. Tela inicial

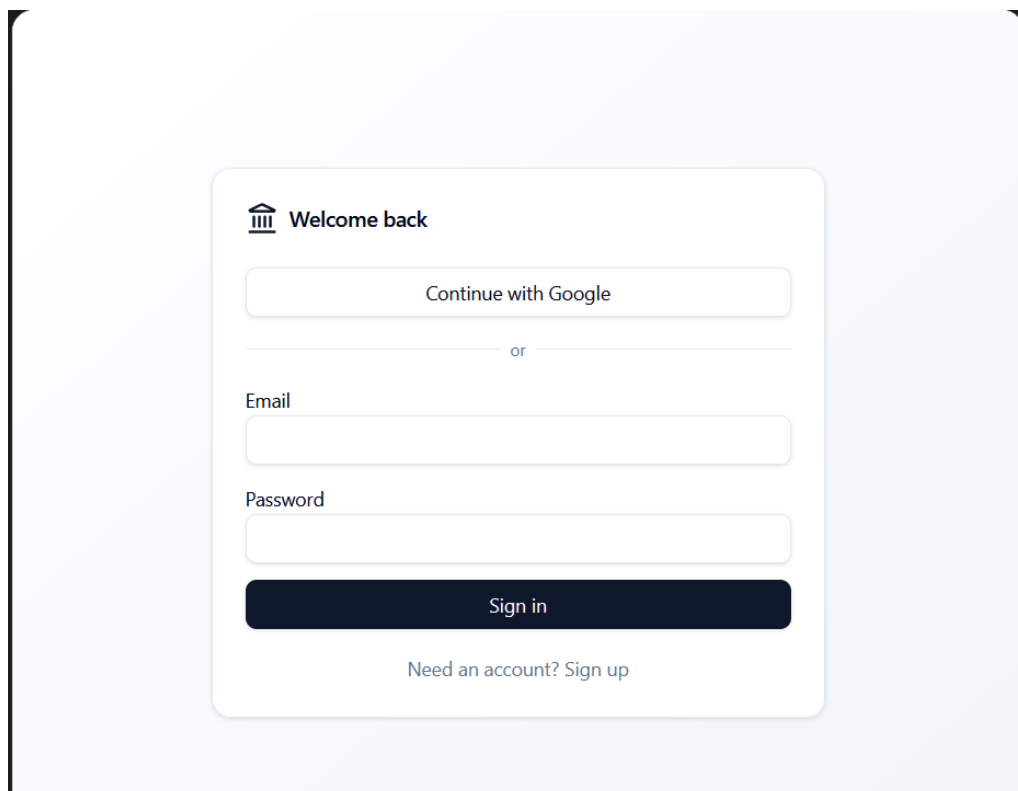


Figura 2. Tela de login

 Open an account

Continue with Google

or

Email

Password

Create account

Already have an account? Sign in

Figura 3. Tela de criação de conta

 OOP Bank Sign out

Accounts + New

No accounts yet. Open one to get started.

Select or open an account.

Figura 4. Criação de contas bancárias

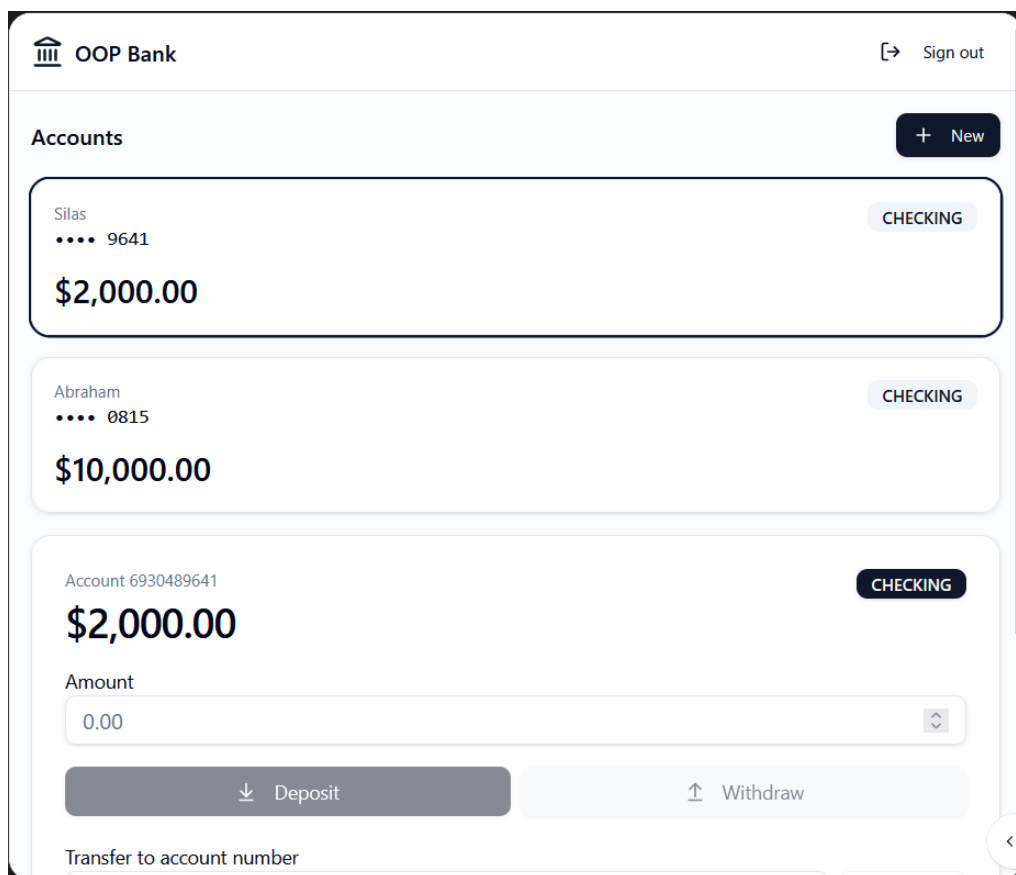


Figura 5. contas bancárias.

6.2.2. Transações Bem-Sucedidas (Depósitos e Transferências)

A integridade financeira do sistema é comprovada durante a execução de transações dinâmicas. A Figura 6 demonstra o painel interno de uma conta, onde é possível realizar e registrar um depósito inicial. A demonstração do fluxo de transferência (a operação mais complexa do sistema) é evidenciada nas Figuras 7 e 8. Na Figura 7, o utilizador inicia uma transferência de \$500 com destino à conta 6930489641. A Figura 8 comprova o sucesso e o sincronismo da operação orquestrada pela classe GerenciadorTransacoes: o histórico registra simultaneamente a saída de capital (TRANSFER OUT de -\$500.00) na conta de origem e a respetiva entrada (TRANSFER IN de +\$500.00) na conta de destino, atualizando os saldos instantaneamente e com precisão.

Abraham
•••• 0815

CHECKING

\$10,000.00

Account 6930489641

CHECKING

\$2,000.00

Amount

0.00

↓ Deposit

↑ Withdraw

Transfer to account number

10-digit account #

↔ Send

Transaction history

DEPOSIT

Opening deposit · 02/07/2026, 23:34:03

+\$2,000.00
bal \$2,000.00

Figura 6. Depósitos, retiradas e transferências.

The image shows a mobile application interface for managing bank accounts. It features three account cards at the top, each with a name, masked card number, and balance. The first card is for 'Silas' with a balance of \$1,500.00. The second is for 'Abraham' with a balance of \$10,500.00. The third card is for 'Account 7510210815' with a balance of \$10,500.00 and includes a 'CHECKING' label. Below the third card is a section for making a deposit or withdrawal. It has a dropdown menu for 'Amount' set to '500', and two buttons: 'Deposit' (dark blue) and 'Withdraw' (light blue). Below this is a 'Transfer to account number' section with a text input field containing '6930489641' and a 'Send' button. At the bottom, there is a 'Transaction history' section with a left-pointing arrow icon.

Silas
•••• 9641
\$1,500.00

Abraham
•••• 0815
\$10,500.00

Account 7510210815
\$10,500.00
CHECKING

Amount
500

DepositWithdraw

Transfer to account number
6930489641
Send

Transaction history

Figura 7. Transferência de 500\$

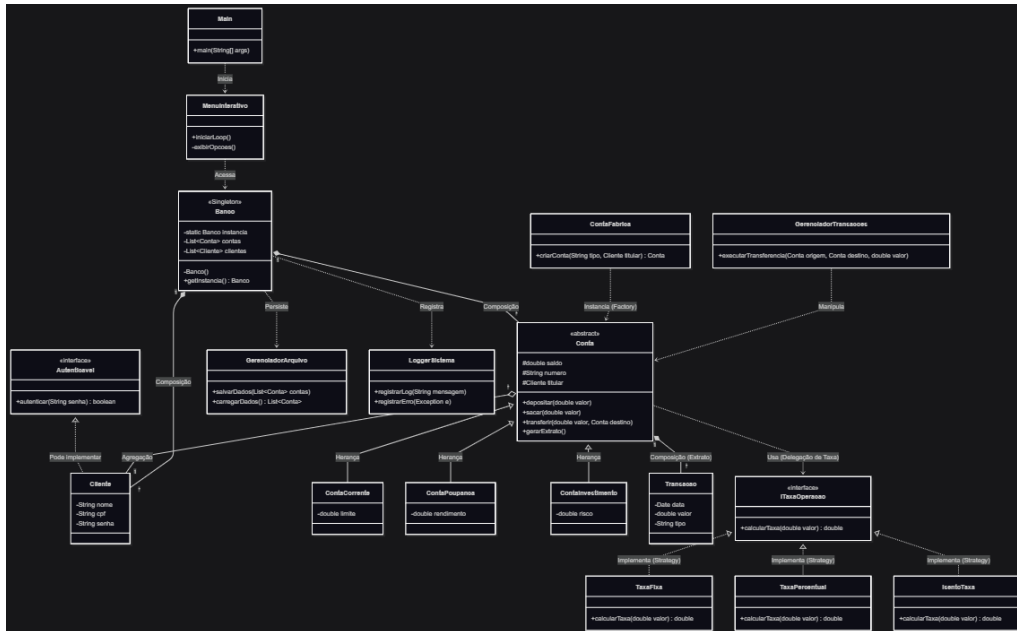
6.2.3. Bloqueio de Operações Inválidas e Segurança

Além das operações bem-sucedidas, o sistema assegura uma proteção rigorosa contra comportamentos anômalos, refletindo o tratamento de exceções previamente modelado. Caso o utilizador tente efetuar um levantamento ou uma transferência de um montante superior ao saldo efetivamente disponível (somado ao limite de crédito, nas Contas Correntes), a interface interceta a ação de imediato. A camada de modelo lança a exceção customizada `SaldoInsuficiente`, a qual é capturada pela interface para bloquear a operação e impedir alterações no saldo. Em vez de causar o encerramento abrupto da aplicação (crash), a interface exhibe um alerta de erro amigável ao utilizador, informando-o da impossibilidade da transação por falta de fundos. O loop do menu permanece ativo, permitindo a correção do valor ou a escolha de uma nova operação.

Transaction history	
TRANSFER OUT To 6930489641 · 02/07/2026, 23:37:52	-\$500.00 bal \$10,000.00
TRANSFER IN From 6930489641 · 02/07/2026, 23:37:16	+\$500.00 bal \$10,500.00
DEPOSIT Opening deposit · 02/07/2026, 23:34:17	+\$10,000.00 bal \$10,000.00

Figura 8. Histórico de transferências.

O diagrama de classes UML final (fornecido nos formatos diagrama.mmd e diagrama.svg) mapeia a arquitetura completa do sistema, dividindo o domínio lógico em pacotes de responsabilidades e ilustrando o rigor da modelagem Orientada a Objetos.



Abaixo estão detalhadas as estruturas de relacionamento que regem a aplicação:

- **Herança (Generalização):** O sistema utiliza herança para estender funcionalidades e reaproveitar código estrutural. A superclasse abstrata *Conta* define os comportamentos e atributos universais do domínio bancário. A partir dela, as subclasses *ContaCorrente*, *ContaPoupanca* e *ContaInvestimento* derivam, herdando a base e especializando atributos únicos (como limite, rendimento e risco) e aplicando o polimorfismo nas regras de negócio financeiras.
- **Implementação de Interfaces (Realização):** Visando baixo acoplamento, contratos abstratos foram estipulados no projeto. As classes *TaxaFixa*, *TaxaPercentual* e *IsentoTaxa* assinam e implementam o contrato da interface *ITaxaOperacao*, consolidando a lógica modular do Padrão Strategy. Paralelamente, a classe *Cliente* adota a interface *Autenticavel*, garantindo o método de login ao perfil.
- **Composição (Relação de Existência Forte):** Foram aplicadas relações de dependência existencial no repositório de dados e nas transações:
 - A classe gerencial *Banco* é inteiramente composta por instâncias de *Conta* e de *Cliente*. Estes elementos dependem do *Banco* como contêiner em memória.
 - Uma *Conta* é estritamente composta por um histórico de instâncias de *Transacao*. O registro e a vida útil do extrato (transações) deixam de fazer sentido caso a conta origem seja destruída.

- **Agregação (Relação de Existência Fraca):** A arquitetura estabelece uma relação de agregação entre Conta e Cliente. Neste cenário lógico, a conta agrega uma referência a um titular (o cliente), contudo, o objeto Cliente é autônomo e pode existir dentro do Banco de Dados independentemente de estar vinculado a uma conta específica.
- **Dependências Arquiteturais e de Padrões:** O acoplamento intencional para fluxo de operações inclui a ContaFabrica, que detém a responsabilidade de instanciar as contas (Padrão Factory), retirando essa carga das demais classes. A interface de entrada Main aciona o MenuInterativo, o qual acessa a base do Banco. O Banco, como elemento mediador (Singleton), depende das ferramentas GerenciadorArquivo e LoggerSistema para orquestrar a gravação em disco e os relatórios. O controle financeiro (GerenciadorTransacoes) depende intrinsecamente das abstrações de Conta para conduzir transferências.

8. Conclusão

O desenvolvimento deste Sistema Bancário cumpriu com êxito os objetivos propostos para a disciplina de Paradigmas de Linguagem de Programação, consolidando na prática os conceitos fundamentais e avançados do Paradigma Orientado a Objetos (POO). A escolha por uma aplicação de gestão financeira permitiu explorar não apenas o encapsulamento, a instanciação de objetos e o relacionamento entre classes, mas também garantir a pontuação extra por meio da implementação de herança, polimorfismo, interfaces e persistência de dados em arquivos .csv e .log.

A reflexão central que este trabalho proporciona é a evidente importância da estruturação em POO para a manutenção e escalabilidade do código. A divisão rigorosa do sistema em pacotes altamente coesos (como modelo, gerenciamento, persistencia e ui) evitou o acoplamento excessivo. Além disso, a adoção de princípios SOLID (como o Princípio da Responsabilidade Única no GerenciadorTransacoes) e Padrões de Projeto (Factory Method para criação de contas, Strategy para cobrança dinâmica de taxas e Singleton para a gestão centralizada no Banco) provou que um sistema bem modelado é capaz de crescer sem que alterações em uma regra de negócio quebrem o funcionamento de outras partes da aplicação.

Como eventuais melhorias futuras e próximos passos para a evolução deste sistema bancário, destacam-se:

- **Integração com Banco de Dados Relacional:** Substituir a atual persistência baseada em arquivos (.csv) por um banco de dados robusto (como PostgreSQL ou MySQL) utilizando JDBC ou um framework ORM (como Hibernate), garantindo maior integridade e segurança transacional.
- **Evolução da Interface Gráfica (GUI):** Migrar de uma interface interativa via console/Swing básica para uma interface gráfica mais rica utilizando JavaFX, ou até mesmo transformá-la em uma API RESTful (com Spring Boot) para ser consumida por um frontend web ou mobile.
- **Segurança e Criptografia:** Implementar algoritmos de hash (como BCrypt) para o armazenamento de senhas dos clientes, além de criptografar os dados sensíveis gerados nos arquivos de exportação do extrato.
- **Concorrência e Multithreading:** Adaptar o GerenciadorTransacoes para lidar com

múltiplas requisições simultâneas de forma segura, simulando o ambiente de alta disponibilidade de um banco real.

Em suma, a prática demonstrou que a Programação Orientada a Objetos vai muito além da simples organização de variáveis e funções; ela é uma ferramenta indispensável para a modelagem de soluções de software resilientes, legíveis e preparadas para as demandas do mundo real.

Nota de Disponibilidade do Código-Fonte:

Para propósitos de replicação, auditoria dos métodos de consenso e implementação *POO* discutidos neste documento, todo o conjunto de arquivos técnicos, incluindo os módulos detalhados, encontra-se aberto e disponível publicamente ***Clique aqui para acessar o repositório.***